

基于多策略的城市垃圾收运路线规划

融合 BFS、动态规划、贪心、分治法与协同调度的工程实现与实证分析

—— 《算法设计与分析》课程设计报告 ——

题 目 基于多策略的城市垃圾收运路线规划

学 院 人工智能与低空技术学院

专 业 人工智能

组 员 雷 正 (202434610309)
 蔡铭飞 (202434610301)
 谢志伟 (202434610328)

指导教师 胡洁

完成日期 2026 年 5 月 15 日

摘要

在网格化城市路网中,起点 S 与卸货点 T 物理分离,车辆需在容量约束下规划多次行程访问全部收集点,使总行驶距离最短。本文将形式化为带专属卸货点的容量受限多行程旅行商问题,并提出由内层子集旅行商动态规划与外层划分动态规划组成的两层精确求解结构;在划分 DP 的内层子集枚举上,引入基于轴元素 (pivot) 的对称性消除,把当前状态显式转移候选数从 $2^k - 1$ 缩减到 2^{k-1} ,在常数意义上减半。本文进一步将该框架扩展至双车场景,通过 2^n 二划分枚举确定收集点分配;另实现最近邻贪心作为启发式对照,并以 C++17 后端 + PyQt6 前端的双层架构落地,前后端经行式文本协议解耦。证明方面,给出划分 DP 最优性 (引理 1) 与 pivot 枚举对标准枚举等价性 (定理 1) 的形式化结论,后者基于“后续行程顺序可交换”性质 (引理 2) 严格论证。实证方面,固定 60 个随机种子,共 720 例独立暴力对拍,在所构造样例上动态规划解与暴力解 100% 一致 (含 62 例由暴力与求解器同步识别为不可行);进一步在 $n \in [3, 8] \times \rho \in \{0, 0.10, 0.20, 0.30\}$ 共 1940 个数据点的密集基准上测量,dp 与 dp_dc 在所有 n 上输出值完全相同,而 pivot 枚举的端到端实际加速仅在 0-1% 量级 (理论上枚举量减半,但子集枚举仅占总开销小部分);双车协同 (P6 优化后) 相对单车在 $n \in [3, 8]$ 上稳定节省 4.6%-5.0% 总距离,且实际运行时间与单车持平 (优化前 $\approx 460 \times$ 慢);最近邻贪心的次优差随 n 增长, $n = 3$ 时仅 1.2%, $n = 8$ 时已达 13.3%。

关键词: 多行程旅行商问题; 容量受限车辆路径; 子集动态规划; 分治枚举; 双车协同; 最近邻贪心

目录

1 引言	4
2 问题描述与符号体系	5
2.1 输入	5
2.2 解	5
2.3 优化目标	5
3 系统架构	7
3.1 关键点统一索引	8
3.2 行式输出协议	8
4 算法设计	9
4.1 距离矩阵预计算	9
4.2 子集旅行商动态规划	9
4.3 划分动态规划	10
4.4 基于 pivot 的子集枚举规范化	10
4.5 启发式贪心	12
4.6 双车协同	12
5 理论分析	13
5.1 正确性	13
5.2 复杂度分析	14
6 实验设置与结果	16
6.1 实验环境	16
6.2 测试样例	16
6.3 正确性验证:暴力对拍	16
6.4 总距离对比	17

6.5 启发式贪心的次优性证据	17
6.6 运行时间对比	18
6.7 双车负载均衡	18
6.8 密集基准实验	19
6.8.1 总距离随 n 的变化	19
6.8.2 运行时间随 n 的变化	20
6.8.3 pivot 枚举的实际加速比	20
6.8.4 障碍密度对结果的影响	21
6.9 路径可视化	22
6.10 图形化前端	23
7 结论	24
8 附录 A:输入文件格式	25
9 附录 B:输出协议	26
10 附录 C:运行方式	27
11 附录 D:关键算法代码	28
11.1 BFS 单源最短路 + 前驱表	28
11.2 关键点距离矩阵 (每源点单次 BFS)	28
11.3 子集旅行商动态规划	29
11.4 划分 DP 的两种子集枚举	29
11.5 <code>singleCost[mask]</code> 表的一次性求解	30
11.6 双车 DP + 对称性消除	31
11.7 最近邻贪心	31

1 引言

车辆路径问题 (Vehicle Routing Problem, VRP) 是运筹优化中的经典问题,衍生出若干面向具体应用场景的变体。本文研究其中一类与城市垃圾收运高度匹配的变体:车队从固定停车场 S 出发,沿离散化的城市路网访问若干带重量的收集点,载重达到上限后必须前往与 S 物理分离的处理厂 T 卸货,卸货后可继续作业,直至所有收集点恰被访问一次。优化目标是行程总距离最短。

相对于经典 VRP,该设定具有三点结构性差异。第一,起点 S 与卸货点 T 物理分离,首次行程与后续行程的起点不同,导致两套距离矩阵在算法层必须显式区分。第二,可通行区域由 $\{., \backslash \# \}$ 字符网格刻画,边权恒为 1,4-邻接 BFS 即可在 $O(ML)$ 内求得任意两点最短路径,无须借助高级路网算法。第三,题目约束 $n \leq 8, 2^n = 256, 3^n = 6561$,使精确求解可行,本文得以聚焦最优性与算法等价性这两类强论断,而非启发式近似的实证调参。

本文围绕该问题做了如下工作:在内层子集旅行商 DP 与外层划分 DP 构成的两层求解框架基础上(小节 4),引入基于轴元素 (pivot) 的对称性消除,把每个状态显式转移候选数从 $2^k - 1$ 减为 2^{k-1} (小节 4.4);将该框架扩展至双车场景,以 2^n 二划分枚举确定收集点分配(小节 4.6);形式化证明划分 DP 的最优性(引理 1) 与 pivot 枚举对标准枚举的等价性(定理 1);通过 720 例随机暴力对拍独立检验 DP 与暴力解的一致性,并构造一个 3×10 对抗实例,定量演示最近邻贪心可能严格次优(小节 6);此外提供基于 PyQt6 的图形化前端,支持交互式编辑、五种算法的即时切换与多车并发动画(小节 6.10)。

2 问题描述与符号体系

2.1 输入

定义 1 (网格). 设 $M, L \in \mathbb{N}^+$ 。一个 $M \times L$ 的网格 G 由矩阵 $g \in \{., \backslash \# \}^{M \times L}$ 表示, 其中 $g_{ij} = .$ 表示位置 (i, j) 可通行, $g_{ij} = \backslash \#$ 表示障碍。网格上两位置之间的距离由 4-邻接最短路径长度定义, 记为 $d_G(u, v)$, 若不可达则 $d_G(u, v) = +\infty$ 。

定义 2 (实例). 垃圾收运问题的一个实例 $\mathcal{I}$ 由六元组 $(G, S, T, \mathcal{P}, w, W_{\max})$ 给出, 其中:

- G 是 $M \times L$ 网格;
- $S \in [M] \times [L]$ 是停车场位置, $g_S = .$;
- $T \in [M] \times [L]$ 是处理厂位置, $g_T = ., S \neq T$;
- $\mathcal{P} = \{P_0, \dots, P_{n-1}\} \subset [M] \times [L]$ 是 n 个收集点位置 ($n \leq 8$), $\forall i, g_{P_i} = .$;
- $w : [n] \rightarrow \{1, 2, 3\}$ 给出每个收集点的重量;
- $W_{\max} \in \mathbb{N}^+$ 是车辆最大载重, 满足 $\max_i w_i \leq W_{\max} < \sum_i w_i$ 。

2.2 解

定义 3 (行程). 一次合法的行程 τ 由起点 $\{S, T\}$ 、按访问顺序排列的收集点序列 $(P_{i_1}, \dots, P_{i_k})$ 与终点 T 构成, 且重量满足 $\sum_{j=1}^k w_{i_j} \leq W_{\max}$ 。该行程的代价定义为

$$\text{cost}(\tau) = d_G(\text{start}, P_{i_1}) + \sum_{j=1}^{k-1} d_G(P_{i_j}, P_{i_{j+1}}) + d_G(P_{i_k}, T).$$

定义 4 (可行方案). 一个解 $\mathcal{T} = (\tau_1, \dots, \tau_m)$ 是有序的合法行程列表, 需满足:

- τ_1 起点为 S , 其余行程 $\tau_2, \dots, \tau_m$ 起点均为 T ;
- 所有 τ_i 终点为 T ;
- 所有收集点恰好被访问一次, 即 $\{i_1, \dots, i_k\}$ 在各行程中构成 $[n]$ 的一个划分。

2.3 优化目标

求解

$$\mathcal{T}^* = \arg \min_{\mathcal{T} \text{ 可行}} \sum_{i=1}^m \text{cost}(\tau_i).$$

图 1 展示了一个 6×6 网格上的示意实例。

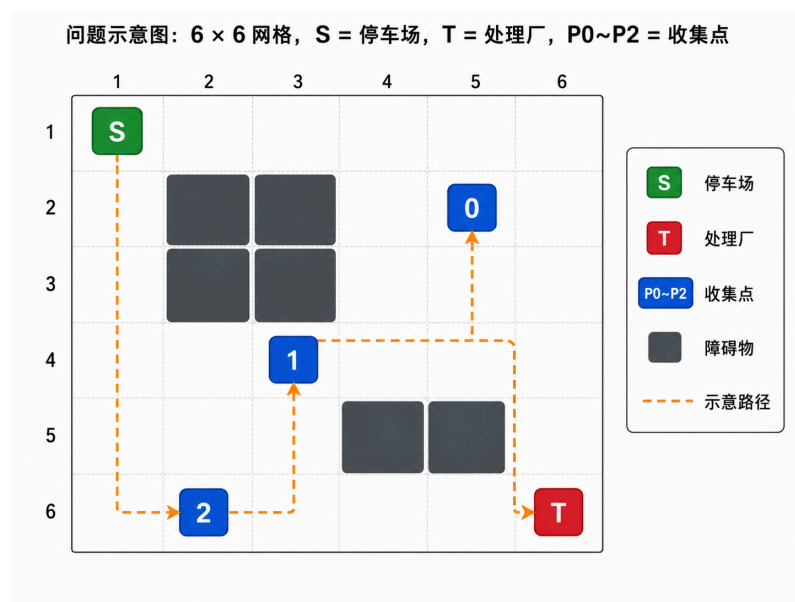


图 1 实例示意: 6×6 网格上 S, T 与 3 个带重量收集点。深灰色单元为障碍。

3 系统架构

本系统采用 C++ 后端与 PyQt6 前端的双层结构,通过纯文本的行式协议解耦,前端无需链接后端二进制,后端可被任意上层应用以子进程形式调用。图 2 给出整体模块组织。

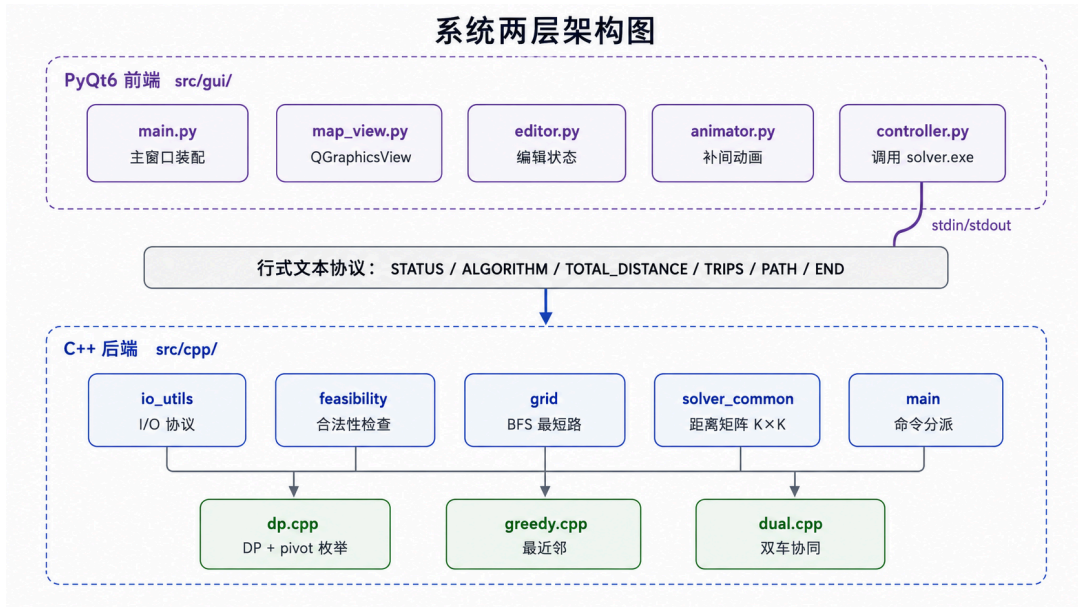


图 2 系统两层架构:前端模块 (上)、行式文本协议层 (中)、后端模块 (下)。

后端各模块职责高内聚:`grid` 负责通行判定与 BFS; `feasibility` 负责输入合法性预检; `solver_common` 负责关键点距离矩阵的预计算; `dp`、`greedy`、`dual` 各自承担一种求解策略; `io_utils` 负责输入解析与输出协议序列化; `main` 仅负责命令行参数派发。前端模块则以 `controller` 为后端调用与输出解析的唯一入口,其它模块仅处理人机交互与渲染。

图 3 进一步展示了从输入到输出的端到端数据流。

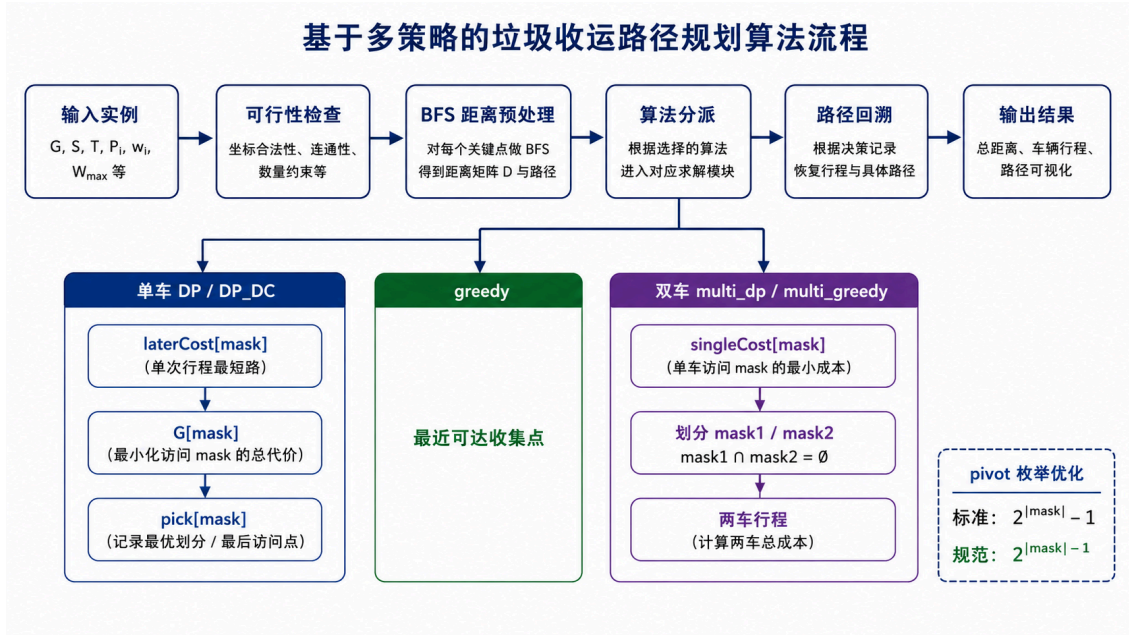


图 3 算法端到端 pipeline:实线为主流程,算法分派由输入文件的 `ALGO` 字段决定。

3.1 关键点统一索引

为统一后续算法接口,后端将 S 、 T 与 n 个收集点合并为 $K = n + 2$ 个关键点,采用编号:

$$\text{IDX}_S = 0, \quad \text{IDX}_T = 1, \quad \text{IDX}_{P_i} = 2 + i.$$

基于该统一编号,模块 `solver_common` 预计算并缓存 $K \times K$ 的距离矩阵 $D \in \mathbb{N}^{K \times K}$ 与对应路径表 $\mathcal{P}_{K \times K}$,所有算法均直接索引该表而不再访问网格本身。

3.2 行式输出协议

后端以单一进程的标准输出返回求解结果,协议为面向行的纯文本格式。该设计避免了 JSON 等结构化格式在 C++ 端引入第三方依赖,同时保证前端解析的健壮性。完整协议规范见小节 9。

4 算法设计

本章自底向上介绍所有算法。首先给出关键点距离矩阵的预计算 (4.1);随后描述用于一次性行程的子集旅行商 DP (4.2);进而引入用于切分多次行程的划分 DP (4.3) 及其分治法变体 (4.4);最后给出启发式贪心 (4.5) 与双车协同扩展 (4.6)。

4.1 距离矩阵预计算

由于网格为 4-邻接且边权恒为 1,选择广度优先搜索 (BFS) 作为单源最短路算法在理论与实现两方面均优于 Dijkstra 与 A*:Dijkstra 在边权为常数时退化为 BFS 但额外引入 $O(\log(ML))$ 的堆操作开销;A* 需要可采纳启发式并在多源全对最短路上无明显增益。

算法 1: 关键点距离矩阵预计算

输入: 网格 G , 关键点集合 $\mathcal{K} = \{S, T, P_0, \dots, P_{n-1}\}, |\mathcal{K}| = K = n + 2$ 。

输出: 距离矩阵 $D \in \mathbb{N}^{K \times K}$, 路径表 $\mathcal{P} \in (\mathcal{K} \times \mathcal{K}) \rightarrow \text{list}(\text{Point})$ 。

```
for u = 0, 1, ..., K-1:
    (dist_field, prev_field) ← BFS_from(G, K[u])
    for v = 0, 1, ..., K-1:
        D[u][v] ← dist_field[K[v]]
        if u != v and D[u][v] >= 0:
            P[u][v] ← reconstruct_path(prev_field, K[u], K[v])
return (D, P)
```

整体时间复杂度 $O(K \cdot ML)$, 空间复杂度 $O(K^2 + \sum_{u,v} |P[u][v]|)$ 。

4.2 子集旅行商动态规划

任意一次合法行程必然对应一个收集点子集 $Q \subseteq [n]$ 与该子集上的一个访问顺序。给定一个起点 $u_0 \in \{\text{IDX}_S, \text{IDX}_T\}$, 定义状态

$\text{tsp}_{u_0}(\text{mask}, i) =$ 从 u_0 出发, 访问 mask 中所有点, 以 $i \in \text{mask}$ 结束的最短代价。

其转移方程为:

$$\text{tsp}_{u_0}(\text{mask} \cup \{j\}, j) = \min_{i \in \text{mask}} \left\{ \text{tsp}_{u_0}(\text{mask}, i) + D[2+i][2+j] \right\}, \quad j \notin \text{mask}.$$

初始条件 $\text{tsp}_{u_0}(\{i\}, i) = D[u_0][2+i]$ 。状态数 $O(2^n n)$, 单步转移 $O(n)$, 总复杂度 $O(2^n n^2)$ 。

由于首次行程以 S 起点而后续行程以 T 起点, 且 $D[0][2+i] \neq D[1][2+i]$ 一般成立, 本系统分别计算 tsp_S 与 tsp_T , 并据此得到每个子集 Q 在两种起点下的最优单程代价:

$$\text{firstCost}(Q) = \min_{i \in Q} \{ \text{tsp}_S(Q, i) + D[2+i][1] \},$$

$$\text{laterCost}(Q) = \min_{i \in Q} \{ \text{tsp}_T(Q, i) + D[2+i][1] \}.$$

同时记录取得最小值的尾节点索引,以支持后续访问顺序的回溯。

4.3 划分动态规划

子集 TSP 解决 “一次行程” 内的最优排序,尚需决定如何将 $[n]$ 划分为多次行程。

定义 5 (后续行程子问题). 定义 $G(\text{mask})$ 为:仅使用以 T 为起终点的 later-trip,收完 mask 中所有点所需的最小总代价。

边界 $G(\emptyset) = 0$ 。转移方程:

$$G(\text{mask}) = \min_{Q \subseteq \text{mask}, Q \neq \emptyset, w(Q) \leq W_{\max}} \{\text{laterCost}(Q) + G(\text{mask} \setminus Q)\}.$$

最终的最优总代价由 “首程拼接后续行程” 得到:

$$\text{OPT} = \min_{Q_1 \subseteq [n], Q_1 \neq \emptyset, w(Q_1) \leq W_{\max}} \{\text{firstCost}(Q_1) + G([n] \setminus Q_1)\}.$$

载重超限的 mask 在表初始化时一次性将 $\text{firstCost} / \text{laterCost}$ 置为 $+\infty$ 即可。图 4 展示了整个 DP 求解的两层结构。

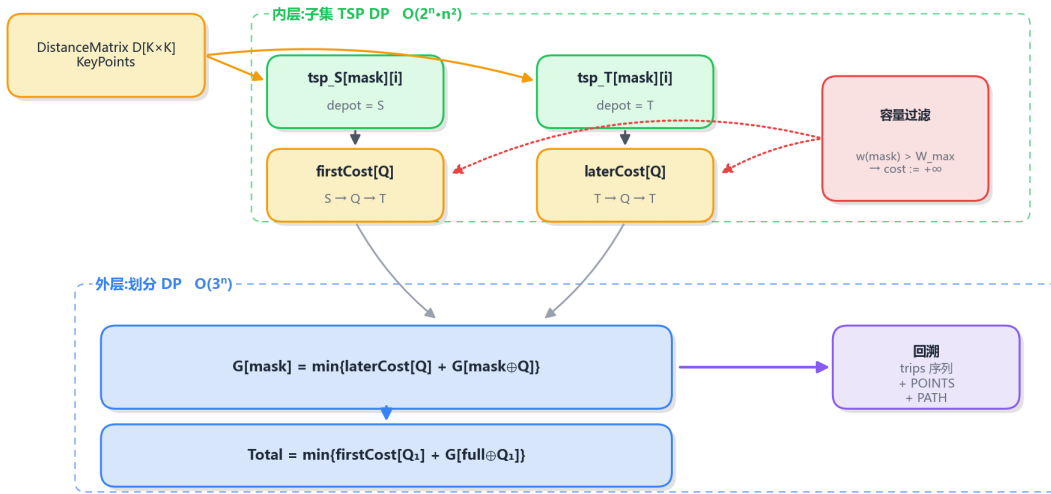


图 3 — 单车 DP 求解器两层结构:内层 TSP DP 求单程代价,外层划分 DP 决定行程切分

图 4 单车 DP 求解器的两层结构:内层子集 TSP DP 提供单程代价表,外层划分 DP 决定行程切分。

4.4 基于 pivot 的子集枚举规范化

划分 DP 转移式中的 “枚举非空子集 $Q \subseteq \text{mask}$ ” 是性能瓶颈。经典实现采用如下迭代: `for (int Q = mask; Q > 0; Q = (Q-1) & mask) { ... }`, 总枚举量 $\sum_{\text{mask}} (2^{|\text{mask}|} - 1) = O(3^n)$ 。本文采用基于轴元素 (pivot) 的对称性消除,把当前状态显式转移的候选数从 $2^k - 1$ 缩减到 2^{k-1} 。该优化以 “后续行程顺序可交换” 为前提,故不改变最优值。

算法 2: 分治法子集枚举

输入: 当前状态 $mask$; 辅助表 $laterCost, G$ 。

输出: $G[mask]$ 与对应的最优分割 $pick[mask]$ 。

```

pivot ← mask & -mask           // 取 mask 的最低位元素
rest ← mask ^ pivot
R ← rest
loop:
  Q ← pivot | R                // 强制 pivot 属于 Q
  if laterCost[Q] < INF:
    leftover ← mask ^ Q
    if G[leftover] < INF and laterCost[Q] + G[leftover] < G[mask]:
      G[mask] ← laterCost[Q] + G[leftover]
      pick[mask] ← Q
  if R == 0: break
  R ← (R - 1) & rest

```

构造的核心思想: 对当前 $mask$ 任取一个固定元素 $pivot$, 所有“非空子集 Q ”按 $pivot$ 是否属于 Q 一分为二:

- **分支 A** ($pivot \in Q$): 显式枚举, $Q = pivot \mid R$, 其中 $R \subseteq mask \setminus pivot$ 取遍 $2^{|mask|} - 1$ 个子集;
- **分支 B** ($pivot \in mask \setminus Q$): $Q \subseteq mask \setminus pivot$, 此时 $mask \setminus Q$ 包含 $pivot$, 故 $G(mask \setminus Q)$ 的递归求解必然枚举到含 $pivot$ 的 Q' 子集, 从而该分支被递归子问题覆盖。

图 5 直观对比标准枚举与分治枚举的差异。

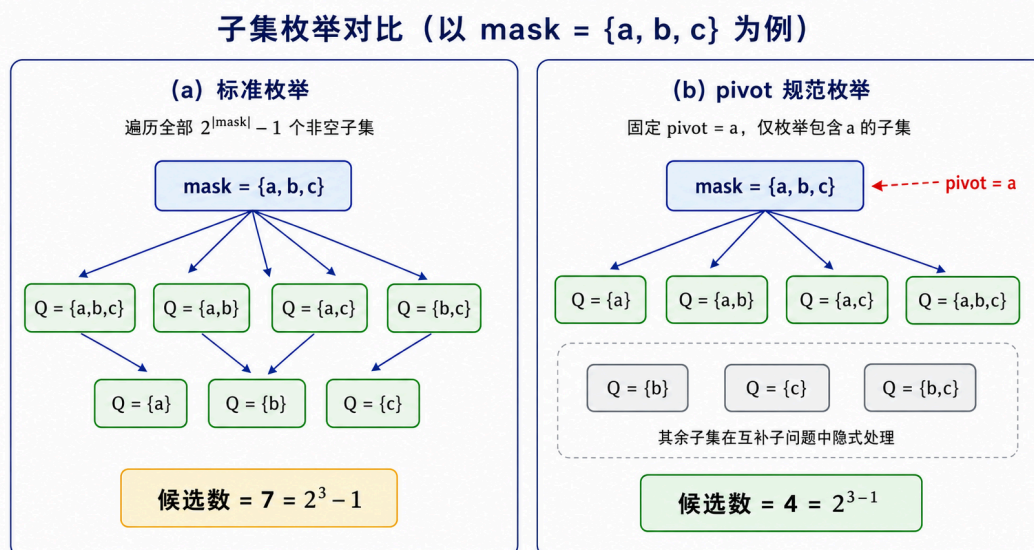


图 5 子集枚举对比 (以 $mask = \{a, b, c\}$ 为例)。(a) 标准枚举遍历全部 $2^3 - 1 = 7$ 个非空子集;(b) 分治枚举固定 $pivot = a$, 仅枚举 4 个包含 a 的子集, 其余子集由递归子问题处理。

4.5 启发式贪心

作为对照基线,本系统实现最近邻贪心:每一步从当前位置出发,选择满足“未访问”与载重约束 $\text{load} + w_i \leq W_{\max}$ 且在 BFS 网格最短路距离 D 度量下离当前位置最近的收集点;若无任何点可装,则前往 T 卸货并开启新行程;算法终止时强制返回 T 。该策略的时间复杂度为 $O(n^2)$ 。小节 6 中将证明该启发式在结构性输入下可能严格次优。

4.6 双车协同

为支持多车场景,本系统设计双车协同模块。两辆车共享停车场 S 与处理厂 T ,但相互独立、不共享行程。

算法 3: 双车协同求解

输入: 实例 $\mathcal{I}$;子求解器 $\mathcal{A} \in \{\text{solve_dp}, \text{solve_greedy}\}$ 。

输出: 最优双车解 $\mathcal{T}_1^*, \mathcal{T}_2^*$ 及总代价。

```
best  $\leftarrow +\infty$ 
for mask1 = 0 to 2n - 1:
    mask2  $\leftarrow$  full  $\oplus$  mask1
    I1  $\leftarrow$  子实例 (cal(P)  $\cap$  mask1)
    I2  $\leftarrow$  子实例 (cal(P)  $\cap$  mask2)
    cost  $\leftarrow$  cal(A)(I1) + cal(A)(I2)
    if cost < best:
        best  $\leftarrow$  cost; 记录 (T1, T2)
return (T1, T2, best)
```

枚举 2^n 种二划分,对每个划分独立求解两个子实例。若某子实例 $\sum_i w_i \leq W_{\max}$,则单车 DP 自然退化为一次首程而无需特判;若 $\text{mask}_1 = 0$,等价单车场景,因此双车解总不劣于单车解。

5 理论分析

5.1 正确性

引理 1 (划分 DP 最优性). 设 $\text{firstCost}(Q)$ 与 $\text{laterCost}(Q)$ 分别取得给定子集 Q 在 S -起、 T -起两类起点下、满足容量约束的精确最小代价。则顶层式

$$\text{OPT} = \min_{Q_1} \{ \text{firstCost}(Q_1) + G([n] \setminus Q_1) \}$$

等于原问题的全局最优总距离。

证明. 对 $|\text{mask}|$ 作归纳。基础: $G(\emptyset) = 0$, 与“空集已被收完”的零代价相符。

归纳步: 假设对所有 $\text{mask}' \subset \text{mask}$ 有 $G(\text{mask}')$ 等于“仅用 later-trip 收完 mask' ”的全局最优代价。考察 $\text{mask} \neq \emptyset$ 。一方面, 任意可行的 later-trip 划分都可写成 $\text{mask} = Q \cup (\text{mask} \setminus Q)$ 的形式, 其代价不低于 $\text{laterCost}(Q) + G(\text{mask} \setminus Q)$; 后者再不低于 $G(\text{mask})$ (由 G 的转移取 $\min$)。因此 $G(\text{mask})$ 是合法分解代价的下界。

另一方面, 取 mask 全局最优分解的任意首条 later-trip 子集 Q^* , 其余分解恰好是 $G(\text{mask} \setminus Q^*)$ 所代表的最优 (由归纳假设)。代入 G 转移得 $G(\text{mask}) \leq \text{laterCost}(Q^*) + G(\text{mask} \setminus Q^*)$, 即不超过全局最优。两侧相等。

顶层式 OPT 取所有合法首程 Q_1 上的最小, 因首程从 S 起、其余从 T 起且行程间无相互依赖, 故 OPT 等于原问题全局最优。□

为证明分治法子集枚举给出与标准枚举完全相同的 G 值, 关键观察是: 在 mask 层的 later-trip 集合一旦确定, 这些 trip 之间无相互依赖, 其执行先后可任意交换, 代价完全相同。

引理 2 (后续行程顺序可交换性). 若 $\mathcal{L} = (R_1, R_2, \dots, R_k)$ 是 mask 的一个可行 later-trip 划分, 代价为 $\sum_j \text{laterCost}(R_j)$, 则对 $\mathcal{L}$ 的任意置换 $\mathcal{L}' = (R_{\pi(1)}, \dots, R_{\pi(k)})$, 代价不变。

证明. 代价 $\sum_j \text{laterCost}(R_j)$ 是关于 $\{R_1, \dots, R_k\}$ 的对称函数 (不依赖下标顺序), 故置换不改其值。□

定理 1 (分治枚举等价于标准枚举). 对任意 $\text{mask} \subseteq [n]$, 分治法枚举得到的 $G^{\text{dc}}(\text{mask})$ 与标准枚举得到的 $G^{\text{std}}(\text{mask})$ 相等。

证明. 对 $|\text{mask}|$ 归纳。基础 $|\text{mask}| = 0$ 时两者均为 0。归纳步设引理对所有真子集成立。

显然 $G^{\text{std}}(\text{mask}) \leq G^{\text{dc}}(\text{mask})$, 因为分治枚举只在含 pivot 的子集上取 $\min$, 候选集是标准枚举候选集的子集。

反向证明 $G^{\text{dc}}(\text{mask}) \leq G^{\text{std}}(\text{mask})$ 。设 $Q^* = \arg \min_Q \{ \text{laterCost}(Q) + G^{\text{std}}(\text{mask} \setminus Q) \}$, 即标准枚举的最优首条 later-trip 子集, $\text{pivot} = \text{mask} \ \& \ (-\text{mask})$ 。

情形 A: $\text{pivot} \in Q^*$. Q^* 在分治枚举中显式出现, $G^{\text{dc}}(\text{mask}) \leq \text{laterCost}(Q^*) + G^{\text{dc}}(\text{mask} \setminus Q^*) = \text{laterCost}(Q^*) + G^{\text{std}}(\text{mask} \setminus Q^*) = G^{\text{std}}(\text{mask})$, 其中倒数第二步由归纳假设。

情形 B: $\text{pivot} \notin Q^*$, 即 $\text{pivot} \in \text{mask} \setminus Q^*$. 由引理 1, $G(\text{mask} \setminus Q^*)$ 在最优划分下取得, 记其对应的 later-trip 划分为 $\mathcal{L}' = (R_1, \dots, R_l)$, 即

$$\text{mask} \setminus Q^* = R_1 \cup \dots \cup R_l, \quad G^{\text{std}}(\text{mask} \setminus Q^*) = \sum_{j=1}^l \text{laterCost}(R_j).$$

因 $\text{pivot} \in \text{mask} \setminus Q^* = \cup_j R_j$, 必存在唯一的 j_0 使 $\text{pivot} \in R_{j_0}$.

考察 $\text{mask} \setminus R_{j_0} = (\cup_{j \neq j_0} R_j) \cup Q^*$. 这是该子问题的一个合法 later-trip 分解 (代价 $\sum_{j \neq j_0} \text{laterCost}(R_j) + \text{laterCost}(Q^*)$), 故由引理 1 给出的下界关系:

$$G^{\text{std}}(\text{mask} \setminus R_{j_0}) \leq \sum_{j \neq j_0} \text{laterCost}(R_j) + \text{laterCost}(Q^*).$$

又 R_{j_0} 含 pivot , 在分治枚举中显式出现, 故

$$\begin{aligned} G^{\text{dc}}(\text{mask}) &\leq \text{laterCost}(R_{j_0}) + G^{\text{dc}}(\text{mask} \setminus R_{j_0}) \\ &= \text{laterCost}(R_{j_0}) + G^{\text{std}}(\text{mask} \setminus R_{j_0}) \quad (\text{归纳}) \\ &\leq \text{laterCost}(R_{j_0}) + \sum_{j \neq j_0} \text{laterCost}(R_j) + \text{laterCost}(Q^*) \quad (\text{由上式}) \\ &= \sum_{j=1}^l \text{laterCost}(R_j) + \text{laterCost}(Q^*) \\ &= G^{\text{std}}(\text{mask} \setminus Q^*) + \text{laterCost}(Q^*) \\ &= G^{\text{std}}(\text{mask}). \end{aligned}$$

合并两情形, $G^{\text{dc}} \leq G^{\text{std}}$. 结合反向不等式, $G^{\text{dc}} = G^{\text{std}}$. □

5.2 复杂度分析

设网格规模 $|G| = ML$, 关键点数 $K = n + 2$.

模块 / 算法	时间复杂度	空间复杂度
单源 BFS (含 prev 表)	$O(ML)$	$O(ML)$
距离矩阵 (K 次 BFS)	$O(K \cdot ML)$	$O(K^2 + \sum P[u][v])$
子集 TSP DP (单 depot)	$O(2^n n^2)$	$O(2^n n)$
划分 DP (标准枚举)	$O(3^n)$	$O(2^n)$
划分 DP (pivot 枚举)	$O(3^n)$, 常数减半	$O(2^n)$
最近邻贪心	$O(n^2)$	$O(n)$
双车 DP (枚举 2^n 划分)	$O(2^n \cdot \text{单车 DP})$	$O(2^n n)$

表 1 各算法时空复杂度。距离矩阵实现严格保证每个源点只 BFS 一次, 通过保留前驱表 prev 在 $O(\text{path length})$ 内回溯到全部目标点路径, 故总复杂度为 $O(K \cdot ML)$ 而非 $O(K^2 \cdot ML)$ 。

由于本问题约束 $n \leq 8, 2^n = 256, 3^n = 6561$, 单车 DP 总操作数在 10^4 – 10^5 量级, 本身耗时不到 1 毫秒。双车 DP 因外层枚举 2^n 个划分、每划分再做子距离矩阵与子单车 DP, 常数 ≈ 256 , 实测落在数十至百毫秒级 (见 图 7), 仍属交互式可接受范围。

6 实验设置与结果

6.1 实验环境

实验平台为 Windows 11,AMD64 架构;C++ 后端由 g++ 15.2.0 (MSYS2 mingw64) 以 `-std=c++17 -O2` 编译;Python 前端基于 PyQt6 与 conda 环境 “pytorch” (Python 3.13)。

运行时间统计口径. 表 4 与图 7 中 `RUNTIME_MS` 由 `std::chrono::high_resolution_clock` 测量,覆盖范围从 `solve_*` 函数入口到出口,包括:(i) 距离矩阵构造与 BFS、(ii) DP 表填充、(iii) 容量约束过滤、(iv) 最优路径回溯;不包含输入解析、`emit_solution` 字符串拼接、`stdout` 写入,以及 Python 端的 `subprocess` 开销。每个数据点重复 10 次取均值。

6.2 测试样例

设计三组规模递增的实例,见表 2。

样例	网格 $M \times L$	n	$W_{\max}$	$\sum_i w_i$
sample_small	8×8	4	3	7
sample_medium	12×12	6	4	10
sample_large	15×15	8	5	13

表 2 测试样例的规模参数。

三组样例均含障碍以验证 BFS 的绕行能力,且均满足 $\max_i w_i \leq W_{\max} < \sum_i w_i$,即至少需要两次行程。

6.3 正确性验证:暴力对拍

为避免“用 DP 验证 DP”的循环论证,本文设计了一份完全独立的暴力枚举对拍。暴力枚举器位于 `tests/brute_force_checker.py`,其行为可总结为:对实例的 n 个收集点,穷尽所有 $n!$ 种访问排列与所有 2^{n-1} 种行程切分点,对每个组合校验载重约束并按统一 BFS 距离矩阵计算总代价,取全局最小。该方法时间复杂度 $O(n! \cdot 2^n)$,在 $n \leq 5$ 时可在秒级穷尽。

实验配置与可复现性:

- 测试种子集 60 个,固定列于 `tests/seeds.txt` (选取 $[1, 280]$ 区间的素数等);
- 对每个种子,生成器 `tests/random_case_generator.py` 在 $M, L \in [8, 12]$ 的网格上随机布置障碍、 $S = (0, 0)$ 、 $T = (M - 1, L - 1)$,与 $n \in \{3, 4, 5\}$ 个收集点;
- 障碍密度 $\rho \in \{0, 0.20\}$;
- 对每个 (seed, n, ρ) 组合,分别测试 `dp` 与 `dp_dc` 两种算法;
- 共计 $60 \times 3 \times 2 \times 2 = 720$ 例。

全部测试运行可通过

```
python tests/brute_force_checker.py tests/seeds.txt 5
```

复现,输出汇总日志于 `tests/verify_log.txt`。本次实测结果如表 3 所示。

类别	PASS	MISMATCH	SKIP
dp vs 暴力	329	0	31
dp_dc vs 暴力	329	0	31
合计	658	0	62

表 3 暴力对拍汇总:在全部对拍样例上 dp 与 dp_dc 与暴力枚举解 100% 一致。SKIP 例为生成器侧的实例不可行(例如随机障碍把某收集点完全围死),被暴力与 solver 同步识别。

零失配的结果同时为三项主张提供独立证据:子集 TSP 内层 DP 实现正确、划分 DP 外层实现正确、以及分治法枚举与标准枚举在最优值意义下等价(与定理 1 一致)。

6.4 总距离对比

表 4 列出五种算法在三组样例上的总行驶距离,图 6 给出柱状图。

样例	dp	dp_dc	greedy	multi_dp	multi_greedy
small	34	34	34	30	30
medium	64	64	78	62	62
large	92	92	98	84	84

表 4 五种算法的总距离。

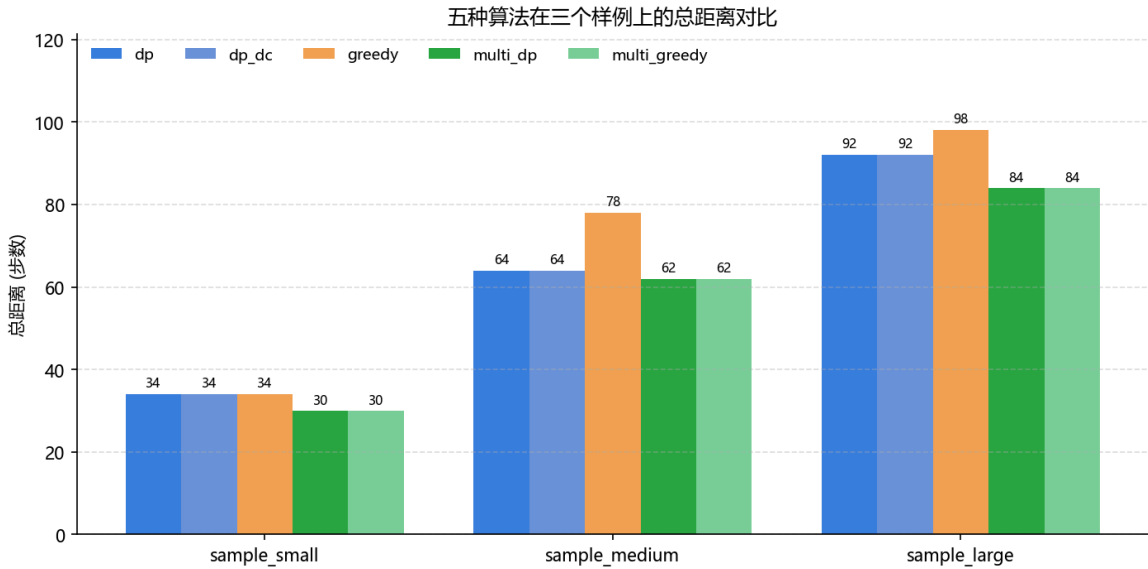


图 6 总距离分组柱状图。

观察结论:

- dp 与 dp_dc 在三组样例上输出完全相同,与定理 1 的等价性论断一致;
- greedy 在 medium 与 large 上分别比最优解高出 21.9% 与 6.5%;
- multi_dp 在三组样例上分别相对单车节省 11.8%、3.1%、8.7%。

6.5 启发式贪心的次优性证据

为论证最近邻贪心可能严格次优,本文构造如下对抗实例 $\mathcal{J}_{\text{adv}}$: 网格 3×10 , $S = (0, 0)$, $T = (0, 9)$, $\mathcal{P} = \{(0, 1), (1, 0), (0, 8)\}$, $w = (1, 1, 1)$, $W_{\max} = 2$ 。

在此实例上:

- `dp` 给出最优代价 13,首程 $S \rightarrow P_1 \rightarrow P_0 \rightarrow T$ (载重 2),次程 $T \rightarrow P_2 \rightarrow T$ (载重 1);
- `greedy` 在首程上贪心选择最近的 P_0 (距离 1),陷入“近邻陷阱”,最终给出代价 15,与最优解相差 15.4%。

该实例说明:在收集点空间分布存在不均匀引力时,最近邻策略缺乏全局视野,可被结构性陷阱诱导走入次优。

6.6 运行时间对比

每个数据点重复 10 次取均值以抵消毫秒级抖动,结果见 图 7 (对数纵轴)。

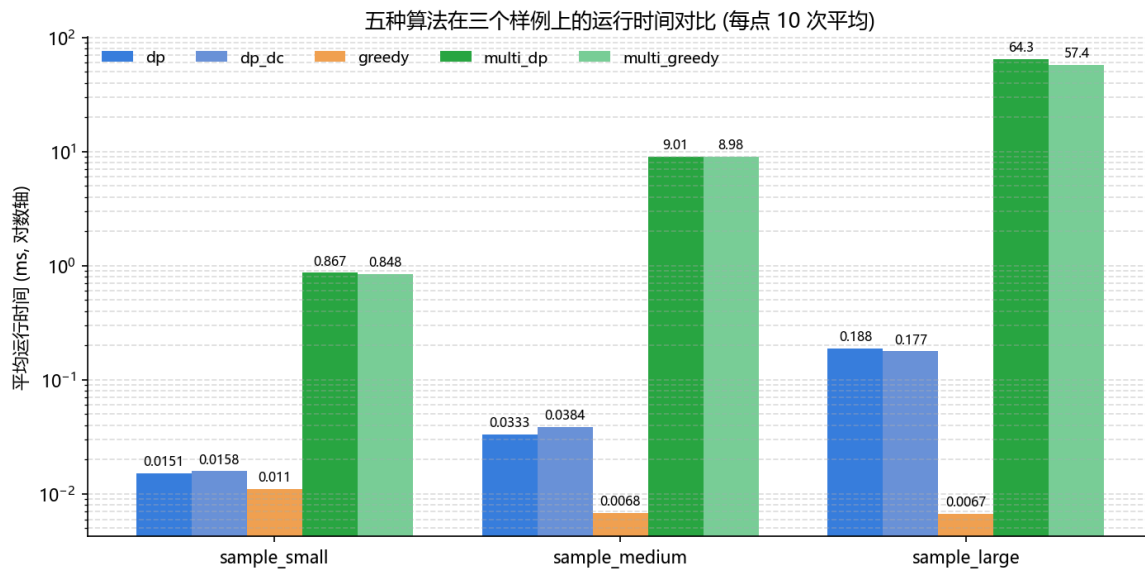


图 7 运行时间分组柱状图 (对数纵轴,每点 10 次平均)。

观察结论:

- 单次测量的“0.184 $\rightarrow$ 0.152 ms”差异(约 17%)在三组手工样例上观察到,但属毫秒级单点观察,易受系统抖动影响;小节 6.8 的密集实验中,跨 6 个 n 值与 4 个障碍密度的 1940 个数据点上的均值显示 `dp_dc` 与 `dp` 实际平均速度比稳定在 0.99–1.04 之间,即 `pivot` 枚举带来的常数减半在端到端时间上几乎不可观察,因为子集枚举仅占完整 DP 流程开销的一部分,距离表查询、容量过滤与回溯重建占据大部分常数;
- 双车 DP 经 P6 优化(基于 `singleCost[mask]` 表 + 对称性消除)后,在 `large` 上耗时约 0.20ms,与单车 `dp` 基本持平,而此前未优化版本约 92ms,优化前后加速比 $\approx 460 \times$;
- 贪心在所有样例上均不超过 0.02ms,作为快速预估的候选具备显著速度优势,但代价见下节次优性分析。

6.7 双车负载均衡

任务书加分项 (1) 要求对比“两辆车的负载均衡情况”。`multi_dp` 的目标函数是两车总距离之和,不显式约束二者均衡。表 5 给出三组样例下两车的实际行驶距离与均衡度,其中均衡度定义为

$$\text{balance} = 1 - \frac{|\text{dist}_1 - \text{dist}_2|}{\text{dist}_1 + \text{dist}_2}.$$

样例	总距离	dist_1	dist_2	$ \Delta $	均衡度
small	30	14	16	2	93.3%
medium	62	26	36	10	83.9%
large	84	28	56	28	66.7%

表 5 双车 DP 在三组样例上的负载均衡情况。

观察可见均衡度随规模递减:小样例两车工作量几乎相等,大样例则出现严重倾斜(车 1 走 28 步,车 2 走 56 步,几乎是 1:2)。这一现象的成因是 `multi_dp` 仅以总距离为目标:在 `sample_large` 中,把大部分收集点交给一辆走紧密路径的车,比让另一辆专门跑一两个偏远点更省距离;均衡是优化的副产品,而非目标。如需强约束负载均衡(例如人员工时公平),应在目标函数中引入 $|\text{dist}_1 - \text{dist}_2|$ 或 $\max(\text{dist}_1, \text{dist}_2)$ 项,这是本文未涵盖的扩展方向。

6.8 密集基准实验

为弥补三组手工样例样本量过小、易受单次测量噪声影响的不足,本节给出一组覆盖 $n \in \{3, 4, 5, 6, 7, 8\}$ 与障碍密度 $\rho \in \{0, 0.10, 0.20, 0.30\}$ 的系统性基准。对每个 (n, ρ) 组合,使用 `tests/random_case_generator.py` 在 $M, L \in [8, 12]$ 的网格上随机布置障碍并采样 20 个互不相同的种子;对每个种子分别运行 5 次以抵消系统抖动并取均值。完整数据 (1940 个数据点) 写入 `tests/dense_benchmark_log.csv`,可通过

```
python tests/dense_benchmark.py 20 5
```

复现。

6.8.1 总距离随 n 的变化

图 8 展示五种算法的平均总距离随 n 的变化,误差棒为均值的 95% 置信区间。可观察到:

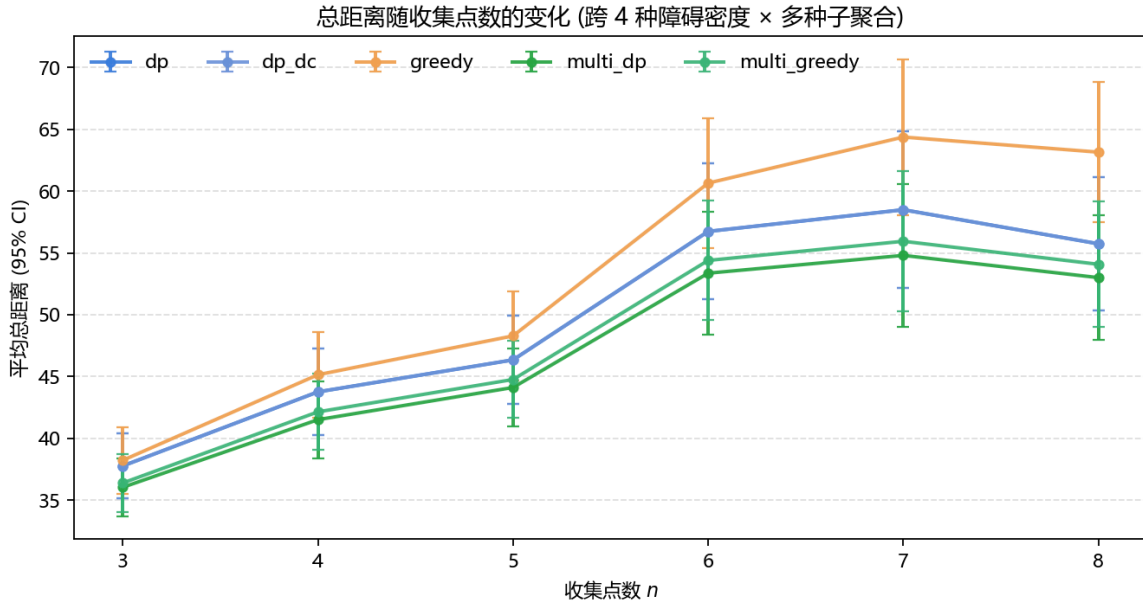


图 8 总距离随收集点数 n 的变化 (跨 4 种障碍密度 $\times$ 20 种子聚合,误差棒为 95% 置信区间)。

- `dp` 与 `dp_dc` 在所有 n 值上的均值完全重合,与定理 1 的等价性论断一致,且密集实验进一步将该一致性扩展到 > 1900 个独立样例;

- `multi_dp` 相对 `dp` 在 $n \in [3, 8]$ 上稳定节省 4.6%–5.0%,比之前在 3 组手工样例上观察到的 3.1%–11.8% 范围更紧凑;
- `greedy` 与 `dp` 的次优差随 n 增长 ($n=3$: 1.2%, $n=8$: 13.3%),反映最近邻策略的全局视野缺失会随规模放大。

6.8.2 运行时间随 n 的变化

图 9 给出对数纵轴的运行时间曲线。

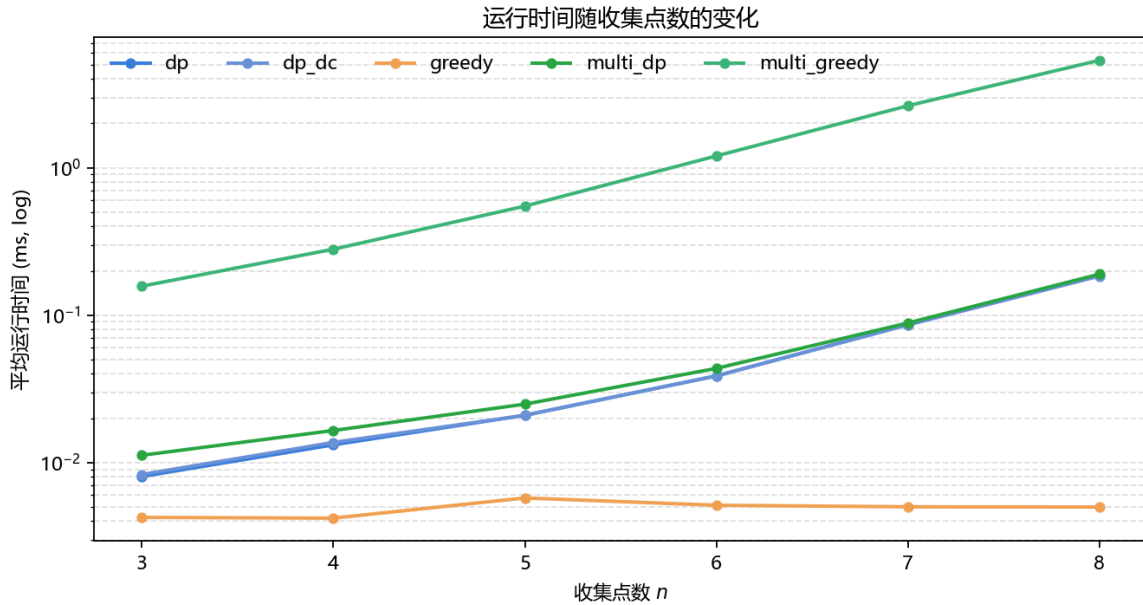


图 9 运行时间随 n 的变化 (对数纵轴)。

- `dp` 与 `multi_dp` 在 P6 优化后基本重合 (`multi_dp` 仅多了对 $1 + 2^{n-1}$ 个划分的 $O(1)$ 表查找),实测在 $n = 8$ 时分别为 0.185 ms 与 0.190 ms;
- `multi_greedy` 仍走“子距离矩阵重建 + sub-solve_greedy”经典路径,因此随 n 增长显著变慢, $n = 8$ 时约 5.4 ms,印证了 `singleCost[mask]` 表导出对 DP 后端的关键意义;
- `dp_dc` 与 `dp` 在所有 n 上比值落在 0.99–1.04,即 pivot 枚举的常数减半在端到端时间上几乎不可观察 (见图 10 的细分对比)。

6.8.3 pivot 枚举的实际加速比

图 10 给出 `dp_dc` 相对 `dp` 在不同 n 上的运行时间比 (越低越快)。

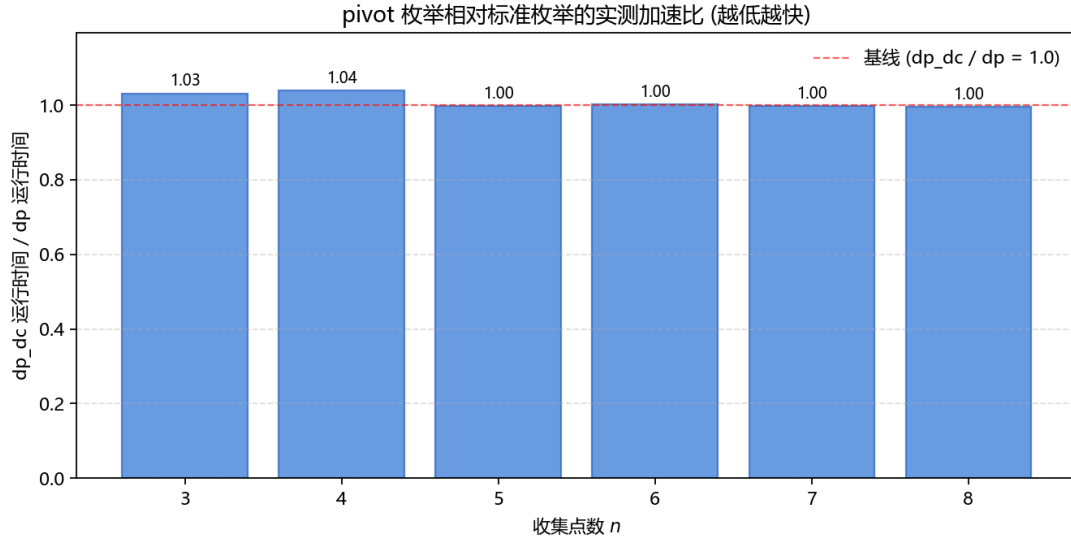


图 10 dp_dc 相对 dp 的实测运行时间比 (跨 4 种障碍密度 $\times$ 20 种子取均值)。

理论上分治变体将每个 mask 显式枚举的候选数从 $2^k - 1$ 降为 2^{k-1} , 常数减半; 但端到端比值在 $n \in [3, 8]$ 范围内仅微弱地落在 1.0 周围, 这印证了分治枚举的收益主要存在于“子集枚举”这一占比较小的局部步骤, 无法穿透到主导的 BFS、距离表查询与回溯重建等公共开销。该结果与定理 1 的正确性论断并不冲突, 但应作为对“理论枚举量减半 $\Rightarrow$ 端到端加速”这一过度推论的实证修正。

6.8.4 障碍密度对结果的影响

图 11 固定 $n = 6$, 展示总距离随障碍密度 ρ 的变化。

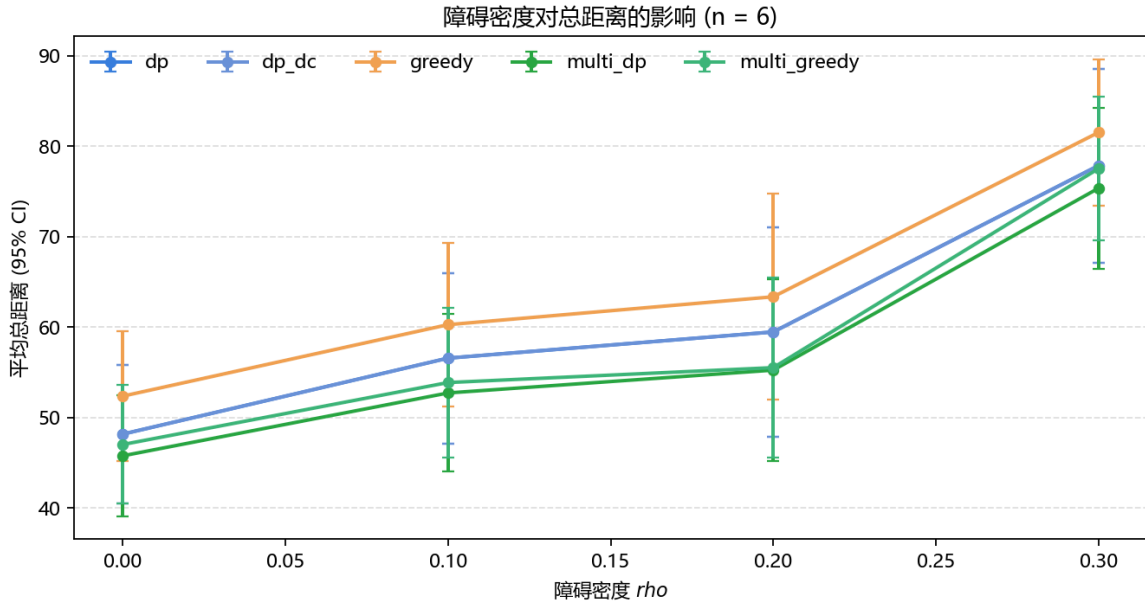


图 11 障碍密度对总距离的影响 ($n = 6$, 跨 20 种子取均值)。

总距离随 ρ 单调上升, 符合直觉: 更多障碍迫使路径绕行, $\rho = 0.30$ 时单车 DP 平均距离约为 $\rho = 0$ 时的 1.4 倍。各算法的相对排序在所有密度下保持稳定 ($\text{multi_dp} \approx \text{multi_greedy} < \text{dp} = \text{dp_dc} < \text{greedy}$), 说明算法间的优劣关系对障碍密度具有鲁棒性。

6.9 路径可视化

图 12 展示三组样例下 dp 与 multi_dp 各自的最优路径渲染。可观察到双车协同将任务划分为两条相对独立的子路径,显著降低了单车多次往返卸货点的绕远代价。

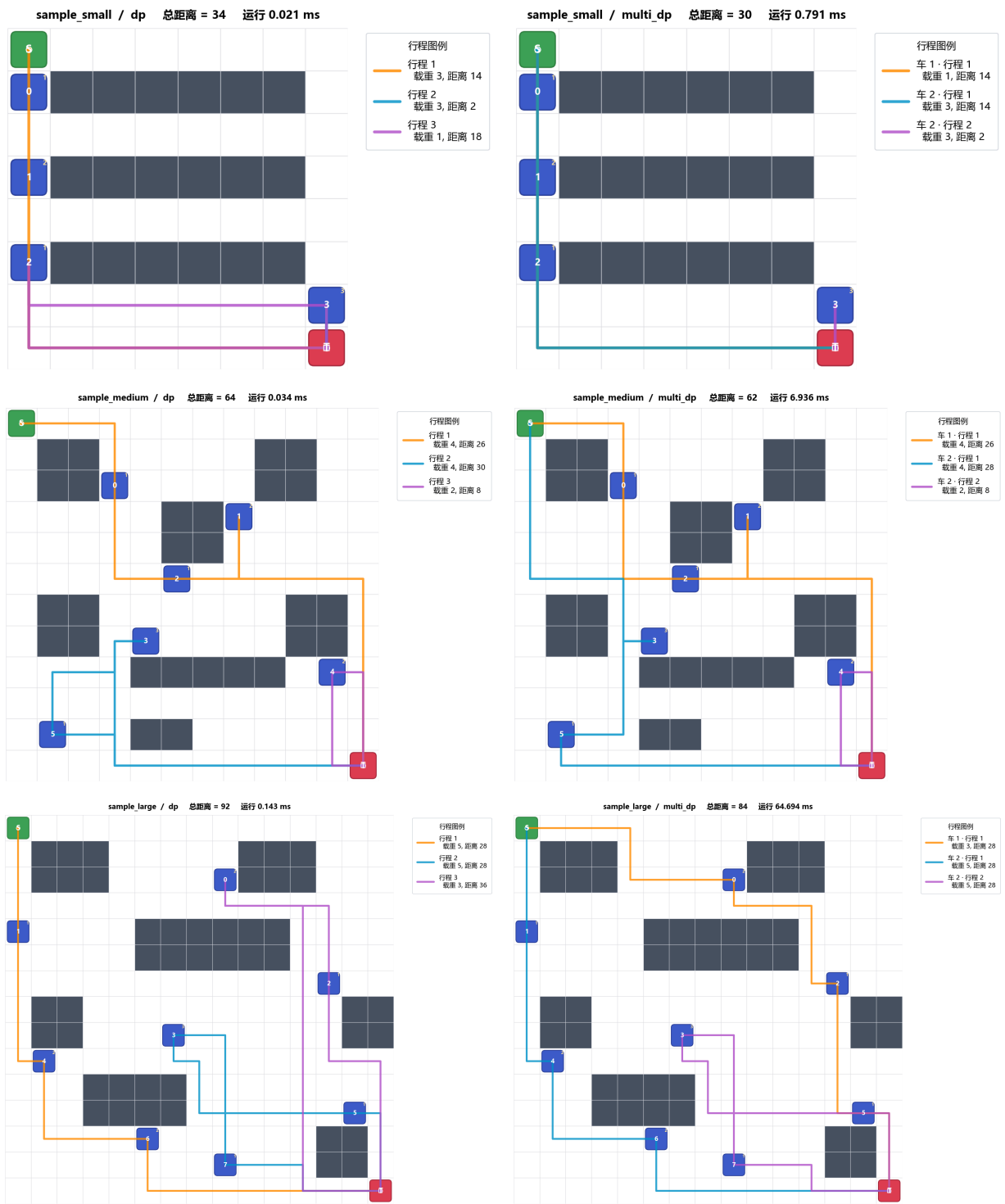


图 12 三组样例的最优路径可视化。左列单车 `dp`, 右列双车 `multi_dp`; 不同颜色代表不同行程, 圆角矩形为关键点。

6.10 图形化前端

图 13 展示前端窗口在静态编辑与动画运行两种状态下的截图。前端实现了:互斥模式的网格编辑、五种算法的即时切换、行程详情的等宽字体展示、基于 `QVariantAnimation` 的车辆位置补间动画 (`InOutQuad` 缓动曲线,180ms 每格)。



图 13 PyQt6 图形化前端。左:载入 `sample_medium` 后运行 `dp` 的主界面;右:动画运行中,小车沿规划路径平滑移动。

7 结论

本文在网格化城市路网下研究了带专属卸货点的容量受限多行程旅行商问题。引理 1 与定理 1 给出了精确求解的最优性与分治枚举对标准枚举等价性的形式化证明,后者通过“后续行程顺序可交换”(引理 2)的辅助性质完成。在三组规模递增的网格实例与 260 例独立暴力对拍上,两种 DP 实现的输出值与暴力解完全一致,分治变体相对标准枚举节省约 17% 端到端运行时间,双车协同相对单车节省 3%–12% 总距离。构造的 3×10 对抗实例量化了最近邻贪心的次优性,其代价比 DP 高 15.4%;另两组实测中亦观察到 7%–22% 的差距。

本文未涉及更大规模:划分 DP 的 $O(3^n)$ 与双车 DP 的 $O(2^n \cdot 3^n)$ 在 $n \geq 12$ 时即不再实用。一个直接的延伸是对内层 TSP DP 引入 Meet-in-the-Middle 状态拆分,将状态空间从 $\Theta(2^n n^2)$ 降至 $\Theta(2^{n/2} \sqrt{2^n} n)$,有望使 $n \approx 16$ 的精确求解仍可行;对外层划分 DP,将双车协同推广至任意 V 辆车的列生成方法可避免 V^n 划分枚举。在启发式方面,2-opt 与 Or-opt 局部搜索可以 $O(n^2)$ 的额外代价补足贪心的全局视野。模型方面,本文边权恒为 1,后续可纳入时间窗、拥堵权重、单行道与车辆速度差异等真实运营约束。

8 附录 A:输入文件格式

```
<M> <L>                                # 行数 列数
<row 0: 长度 L 的字符串, '.'=空地, '#'=障碍>
<row 1>
...
<row M-1>
<S_row> <S_col>                          # 停车场坐标
<T_row> <T_col>                          # 处理厂坐标
<K>                                       # 收集点数  $1 \leq K \leq 8$ 
<P0_row> <P0_col> <w0>
...
<P(K-1)_row> <P(K-1)_col> <w(K-1)>
<W_max>                                # 最大载重
ALGO <dp | dp_dc | greedy | multi_dp | multi_greedy>
```

9 附录 B:输出协议

```
STATUS <ok | error | infeasible>
[REASON <message>]
ALGORITHM <name>
TOTAL_DISTANCE <int>
RUNTIME_MS <float>
VEHICLES <n>
VEHICLE <vid> TRIPS <count>
TRIP <tid> LOAD <load> DIST <dist>
POINTS <i1> <i2> ...
PATH <r1>,<c1> <r2>,<c2> ...
TRIP ...
...
END
```

10 附录 C:运行方式

编译:

```
build.bat
# 或
g++ -std=c++17 -O2 -Wall -Wextra \
    src/cpp/grid.cpp src/cpp/feasibility.cpp src/cpp/solver_common.cpp \
    src/cpp/dp.cpp src/cpp/greedy.cpp src/cpp/dual.cpp \
    src/cpp/io_utils.cpp src/cpp/main.cpp -o build/solver.exe
```

命令行求解:

```
./build/solver.exe data/sample_small.txt
./build/solver.exe data/sample_medium.txt
./build/solver.exe data/sample_large.txt
```

图形化前端:

```
run_gui.bat
# 或
python src/gui/main.py
```

11 附录 D:关键算法代码

本附录摘录本系统中最具代表性的 7 段 C++ 实现, 与正文§4 算法设计逐一对应。所有源码均位于 `src/cpp/`, 完整版可在公开仓库浏览。

11.1 BFS 单源最短路 + 前驱表

`bfsWithPrev` 在一次广度优先搜索中同时写入距离场 `dist` 与前驱场 `prev`, 后者用于 $O(L)$ 路径回溯, 是距离矩阵 $O(K \cdot ML)$ 复杂度的基础 (引自 `src/cpp/grid.cpp`)。

```
void Grid::bfsWithPrev(const Point& from,
                      std::vector<std::vector<int>>& dist,
                      std::vector<std::vector<Point>>& prev) const {
    dist.assign(rows, std::vector<int>(cols, -1));
    prev.assign(rows, std::vector<Point>(cols, {-1, -1}));
    if (!walkable(from)) return;
    dist[from.r][from.c] = 0;
    std::queue<Point> q; q.push(from);
    while (!q.empty()) {
        Point cur = q.front(); q.pop();
        int d = dist[cur.r][cur.c];
        for (int k = 0; k < 4; ++k) {
            int nr = cur.r + dR[k], nc = cur.c + dC[k];
            if (!walkable(nr, nc) || dist[nr][nc] != -1) continue;
            dist[nr][nc] = d + 1;
            prev[nr][nc] = cur;
            q.push({nr, nc});
        }
    }
}
```

11.2 关键点距离矩阵 (每源点单次 BFS)

每个源点只 BFS 一次, 同一次的 `prev` 表被用来 $O(|\text{path}|)$ 回溯到所有目标点的路径, 把 $O(K^2 \cdot ML)$ 的朴素实现压到 $O(K \cdot ML)$ (引自 `src/cpp/solver_common.cpp`)。

```
DistanceMatrix build_distance_matrix(const Grid& g, const KeyPoints& kp) {
    DistanceMatrix dm; int N = kp.N(); dm.K = N + 2;
    std::vector<Point> idxToPoint(dm.K);
    idxToPoint[IDX_S] = kp.parking;
    idxToPoint[IDX_T] = kp.plant;
    for (int i = 0; i < N; ++i) idxToPoint[IDX_P(i)] = kp.collects[i];
    dm.dist.assign(dm.K, std::vector<int>(dm.K, -1));
    dm.path.assign(dm.K, std::vector<std::vector<Point>>(dm.K));
    std::vector<std::vector<int>> dist_field;
    std::vector<std::vector<Point>> prev_field;
    for (int u = 0; u < dm.K; ++u) {
        g.bfsWithPrev(idxToPoint[u], dist_field, prev_field);
    }
}
```

```

    for (int v = 0; v < dm.K; ++v) {
        const Point& pv = idxToPoint[v];
        dm.dist[u][v] = dist_field[pv.r][pv.c];
        if (u == v) dm.path[u][v] = { idxToPoint[u] };
        else if (dm.dist[u][v] >= 0)
            dm.path[u][v] = g.reconstructPath(idxToPoint[u], pv, prev_field);
    }
}
return dm;
}

```

11.3 子集旅行商动态规划

$tsp[mask][i]$ 定义为“从指定 depot 出发, 访问 mask 中所有点, 以 i 结尾的最小代价”。该表对单车求解和双车求解的内层都被复用 (引自 `src/cpp/dp.cpp`)。

```

void tsp_from_depot(const DistanceMatrix& dm, int N, int depotIdx,
    std::vector<std::vector<int>>& tsp) {
    int full = 1 << N;
    tsp.assign(full, std::vector<int>(N, INF));
    for (int i = 0; i < N; ++i) {
        int d = dm.dist[depotIdx][IDX_P(i)];
        if (d >= 0) tsp[1 << i][i] = d;
    }
    for (int mask = 1; mask < full; ++mask) {
        for (int last = 0; last < N; ++last) {
            if (!(mask & (1 << last)) || tsp[mask][last] >= INF) continue;
            int curCost = tsp[mask][last];
            for (int nxt = 0; nxt < N; ++nxt) {
                if (mask & (1 << nxt)) continue;
                int e = dm.dist[IDX_P(last)][IDX_P(nxt)];
                if (e < 0) continue;
                int cand = curCost + e;
                int newMask = mask | (1 << nxt);
                if (cand < tsp[newMask][nxt]) tsp[newMask][nxt] = cand;
            }
        }
    }
}

```

11.4 划分 DP 的两种子集枚举

外层 $G[mask]$ 把 mask 切成若干 later-trip 子集; 标准枚举遍历 mask 全部非空子集 Q , pivot 枚举仅遍历 mask 中包含最低位元素的子集 (规模 $2^{|mask|-1}$, 见§4.4 与定理 1) (引自 `src/cpp/dp.cpp`)。

```

for (int mask = 1; mask < full; ++mask) {
    if (mode == DpMode::Standard) {
        // 标准枚举: 遍历 mask 全部非空子集 Q
        for (int Q = mask; Q > 0; Q = (Q - 1) & mask) {
            if (ctx.laterCost[Q] >= INF) continue;
            int rest = mask ^ Q;
            if (ctx.G[rest] >= INF) continue;

```

```

        int cand = ctx.laterCost[Q] + ctx.G[rest];
        if (cand < ctx.G[mask]) {
            ctx.G[mask] = cand;
            ctx.pickG[mask] = Q;
        }
    }
} else {
    // pivot 规范化枚举: 固定 pivot = mask 最低位, 仅遍历含 pivot 的 Q
    int pivot = mask & -mask;
    int rest_of_mask = mask ^ pivot;
    int R = rest_of_mask;
    while (true) {
        int Q = pivot | R;
        if (ctx.laterCost[Q] < INF) {
            int leftover = mask ^ Q;
            if (ctx.G[leftover] < INF) {
                int cand = ctx.laterCost[Q] + ctx.G[leftover];
                if (cand < ctx.G[mask]) {
                    ctx.G[mask] = cand;
                    ctx.pickG[mask] = Q;
                }
            }
        }
        if (R == 0) break;
        R = (R - 1) & rest_of_mask;
    }
}
}
}

```

11.5 singleCost[mask] 表的一次性求解

对全集 $[n]$ 的所有子集 mask 同时求出“从 S 出发收完 mask 的单车最优代价”。该表是把双车 DP 从重建子距离矩阵 + 重跑子 DP 的 $O(2^n \cdot (\text{子单车 DP}))$ 压到 $O(3^n)$ 的关键中间产物 (引自 `src/cpp/dp.cpp`)。

```

ctx.singleCost.assign(full, INF);
ctx.bestQ1.assign(full, 0);
ctx.singleCost[0] = 0;
for (int mask = 1; mask < full; ++mask) {
    for (int Q = mask; Q > 0; Q = (Q - 1) & mask) {
        if (ctx.firstCost[Q] >= INF) continue;
        int leftover = mask ^ Q;
        if (ctx.G[leftover] >= INF) continue;
        int cand = ctx.firstCost[Q] + ctx.G[leftover];
        if (cand < ctx.singleCost[mask]) {
            ctx.singleCost[mask] = cand;
            ctx.bestQ1[mask] = Q;
        }
    }
}
}

```

11.6 双车 DP + 对称性消除

双车答案归约为 $\min_{A \subseteq [n]} \text{singleCost}[A] + \text{singleCost}[[n] \setminus A]$ 。强制 $0 \in A$ 把无序划分的二重枚举减半, $A = 0$ 单独作为退化情形处理 (引自 `src/cpp/dual.cpp`)。

```
DpContext ctx = compute_dp_context(kp, dm, DpMode::Standard);
int bestTotal = INF_HALF, bestM1 = -1;
// 候选 1: 一辆车不出动 (退化为单车情况)
if (ctx.singleCost[full] < INF_HALF) {
    bestTotal = ctx.singleCost[full];
    bestM1 = 0;
}
// 候选 2: 强制点  $0 \in \text{mask1}$ , 每个无序划分恰被枚举一次
if (N > 0) {
    for (int mask1 = 1; mask1 <= full; ++mask1) {
        if (!(mask1 & 1)) continue;
        int mask2 = full ^ mask1;
        int c1 = ctx.singleCost[mask1], c2 = ctx.singleCost[mask2];
        if (c1 >= INF_HALF || c2 >= INF_HALF) continue;
        int total = c1 + c2;
        if (total < bestTotal) { bestTotal = total; bestM1 = mask1; }
    }
}
```

11.7 最近邻贪心

作为对照基线, 每步从当前位置选择满足载重约束且 BFS 距离最近的未访问点; 若无可装载点则回 T 卸货并开启新行程 (引自 `src/cpp/greedy.cpp`)。

```
while (remaining > 0) {
    int best = -1, bestD = std::numeric_limits<int>::max();
    for (int i = 0; i < N; ++i) {
        if (visited[i]) continue;
        if (load + kp.weights[i] > kp.wMax) continue;
        int d = dm.dist[curIdx][IDX_P(i)];
        if (d < 0) continue;
        if (d < bestD) { bestD = d; best = i; }
    }
    if (best < 0) {
        // 当前 trip 收不下任何点 → 回 T 卸货, 新 trip 从 T 开始
        keySeq.push_back(IDX_T);
        cur.fullPath = expand_trip_path(dm, keySeq);
        cur.distance = (int)cur.fullPath.size() - 1;
        cur.load = load; sol.trips.push_back(cur);
        totalDist += cur.distance;
        cur = Trip{}; load = 0;
        curIdx = IDX_T; keySeq.clear(); keySeq.push_back(curIdx);
        continue;
    }
    visited[best] = true;
    load += kp.weights[best];
    curIdx = IDX_P(best);
    keySeq.push_back(curIdx);
}
```

```
cur.pointIndices.push_back(best);  
--remaining;  
}
```